Lab Assignment 2

Cafeteria Order Management System

CSC241 – Object-Oriented Programming | BS Artificial Intelligence (BAI) | Java | 10 Marks | In-Lab

CLO4 alignment:

CLO4 (assignment alignment): Implement and demonstrate a Java program using object-oriented design principles, including interacting classes, composition, aggregation, constructors, and one-dimensional arrays of objects.

Students demonstrate CLO4 through working Java source code, class collaboration, correct object relationships, and an in-lab execution of the program.

Learning outcomes

Design cooperating Java classes using encapsulation, constructors, constructor overloading/chaining, enum, composition, references to existing objects, and traversal of a one-dimensional object array.

Restrictions: No two-dimensional arrays, ArrayList, collection classes, inheritance, abstract classes, or interfaces.

Scenario

A university cafeteria accepts customer orders. Each FoodItem has a code, name, price, and category. A Customer exists independently and may place an Order. Each Order holds up to a fixed number of OrderItem entries in one OrderItem[] array. Each entry represents an existing FoodItem and its ordered quantity. The Order creates its OrderItem objects when new entries are added and computes the total bill.

Required classes and methods

FoodCategory (enum): MAIN_COURSE, DRINK, SNACK, DESSERT.

FoodItem: private itemCode:int, name:String, price:double, category:FoodCategory; constructors FoodItem(int,String,double) and FoodItem(int,String,double,FoodCategory); displayItem():void; getters needed for calculations and search. The shorter constructor must chain to the longer one.

Customer: private customerId:int, name:String; Customer(int,String); displayCustomer():void.

OrderItem: private item:FoodItem, quantity:int; OrderItem(FoodItem,int); getSubTotal():double; displayOrderItem():void. It stores a reference to an already-created FoodItem.

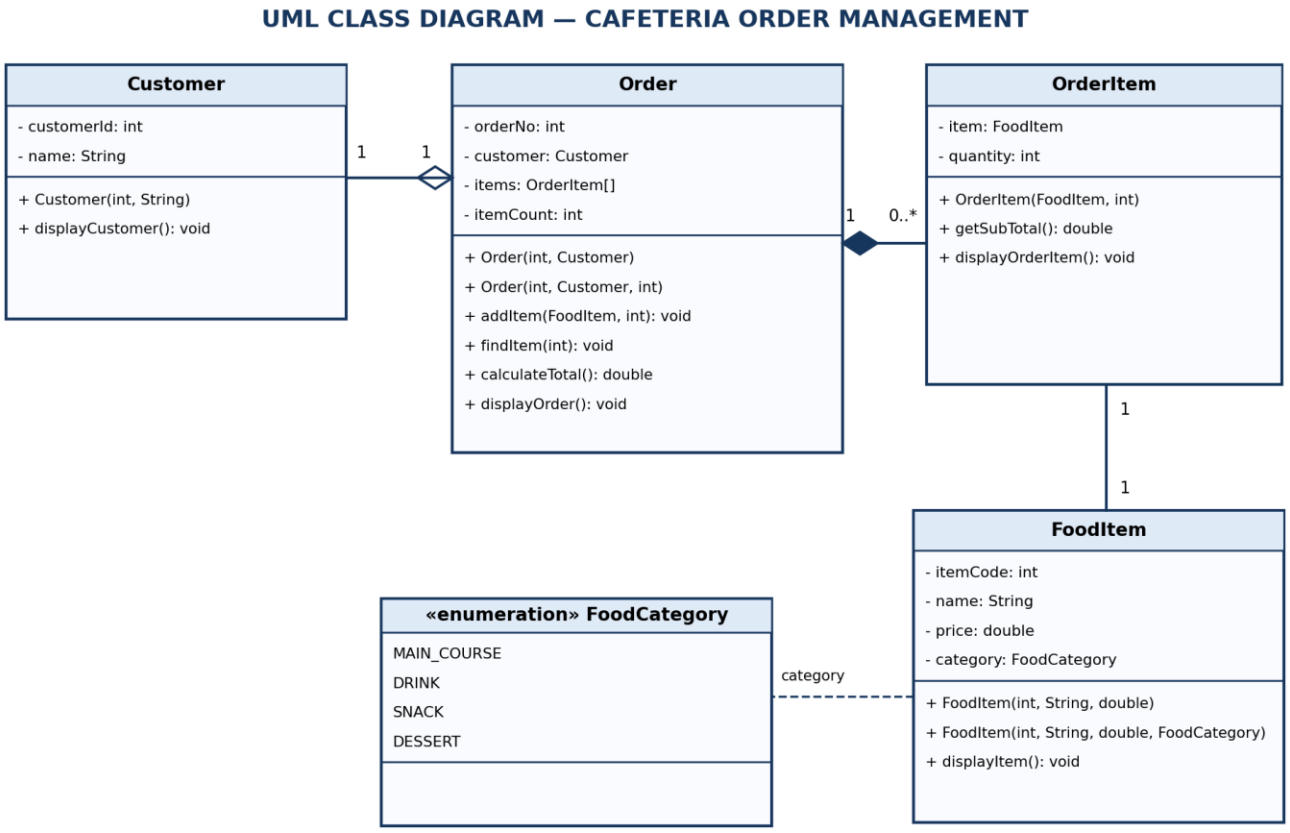
Order: private orderNo:int, customer:Customer, items:OrderItem[], itemCount:int; Order(int,Customer) and Order(int,Customer,int); addItem(FoodItem,int):void; findItem(int):void; calculateTotal():double; displayOrder():void. Default capacity must be 5, using this(...) chaining.

CafeteriaDemo: main(String[]): create at least four foods from different categories, a customer, and an order; add at least three distinct items; print the order and search for an existing item and a missing item.

Behavior and validation

- Store order entries in a one-dimensional OrderItem[]; maintain itemCount and iterate only over populated positions.
- Order.addItem(...) must instantiate a new OrderItem internally. Reject nonpositive quantities and additions when the order is full, with a clear message.
- getSubTotal() returns FoodItem price × quantity; calculateTotal() sums subtotals through array traversal.
- findItem(int itemCode) searches the occupied entries and prints found item details or “Item not found”.
- Do not put all processing in main(); use the methods of the appropriate classes.

UML class diagram



Notation: filled diamond = composition (Order owns OrderItem); unfilled diamond = shared aggregation (Order refers to Customer); plain association = OrderItem refers to an existing FoodItem. Multiplicities show one customer per order and zero to many order entries.

Provided main file — CafeteriaDemo.java

```
public class CafeteriaDemo {
    public static void main(String[] args) {
        FoodItem burger = new FoodItem(101, "Chicken Burger", 650,
                                       FoodCategory.MAIN_COURSE);
        FoodItem drink = new FoodItem(102, "Soft Drink", 150,
                                      FoodCategory.DRINK);
        FoodItem fries = new FoodItem(103, "French Fries", 250,
                                      FoodCategory.SNACK);
        FoodItem dessert = new FoodItem(104, "Ice Cream", 300,
                                        FoodCategory.DESSERT);

        Customer customer = new Customer(1, "Ali");
        Order order = new Order(5001, customer);

        order.addItem(burger, 2);
        order.addItem(drink, 1);
        order.addItem(fries, 2);

        order.displayOrder();
        System.out.println("\nSearching for item 102:");
        order.findItem(102);
        System.out.println("\nSearching for item 999:");
        order.findItem(999);
    }
}
```

Students must implement FoodCategory, FoodItem, Customer, OrderItem, and Order so that this provided main class compiles and runs without modification. The dessert object is deliberately created but not added to the order.

Expected console output (format may vary; values must match)

```
==========================================
         CAFETERIA ORDER
==========================================
Order No: 5001
Customer ID: 1
Customer Name: Ali
------------------------------------------
Item                Qty    Price    Subtotal
Chicken Burger        2      650        1300
Soft Drink            1      150         150
French Fries          2      250         500
------------------------------------------
Total Bill: Rs. 1950

Searching for item 102:
Item Found: Soft Drink | Quantity: 1

Searching for item 999:
Item not found
```

Marking scheme (10 marks)

Assessment criterion	Marks
Class design: attributes, encapsulation, constructors, chaining, enum and object relationships	3
Correct 1-D OrderItem array and methods: add, search, calculate total and capacity handling	5
Successful execution, expected output and source-code ZIP submission	2
Total	10

Submission

Complete and demonstrate the program during the assigned lab session. Submit exactly one ZIP archive containing the required Java source files. Do not upload screenshots or compiled .class files as substitutes for source code.

Submission form: https://forms.gle/xQdQp4hmyaZk2sya8

1. Select your registration number and name from the Student Registration dropdown.

2. Enter the individual BAI-LA2 access code sent to your university email address.

3. Create a ZIP archive named SP26-BAI-001_Lab2.zip (replace 001 with your registration suffix).

4. Include FoodCategory.java, FoodItem.java, Customer.java, OrderItem.java, Order.java, and CafeteriaDemo.java in the archive.

5. Upload the ZIP using the form, submit before the deadline announced in the lab, and confirm your submission.

6. Be prepared to execute the code and explain the composition, aggregation, and array-based methods to the instructor.

Important: The access code is student-specific. Do not share it. Selection and entry of a code do not, by themselves, verify ownership until responses are checked.